EXPERIMENT 6B: - TIME SERIES DATA ANALYSIS AND FORECAST

NAME: MOHAMMED FAYYAD
ROLL NO: 241801165

Learning Objective

The objective of this experiment is to enable students to understand and implement the fundamental concepts of time series data analysis and forecasting using Python.

Problem Statement

An airline company maintains monthly records of the number of passengers travelling by air.

The company wants to understand historical passenger trends and seasonal patterns and use

this information to forecast future passenger demand.

As a Data Scientist, analyse the historical monthly airline passenger data, determine whether the time series is stationary, apply appropriate transformations when necessary, build an ARIMA forecasting model, predict passenger numbers for the test period, and evaluate the accuracy of the forecasts.

Tasks to Perform

1. Download airline-passenges dataset from
1. https://github.com/jbrownlee/Datasets/blob/master/airline-passengers.csv
2. Read the CSV with parse_dates and index_col so the time index becomes a DatetimeIndex.
3. Explore the Dataset.
4. Plot the series to inspect trend/seasonality/outliers.
5. Do Stationarity check by running Augmented Dickey-Fuller (ADF) test.
6. If the series is non-stationary, convert it to stationary by differencing the series.
7. Re-run ADF on the differenced series and visualize the new dataframe.
8. If still non-stationary, consider another difference (d=2) or transformation (log) before differencing.
10. Split the dataset for training and testing on original series.
11. Fit ARIMA model on training set. d value comes from Step 8 (how many differences you applied).
12. Perform forecasts on the test dataset.
13. Compute metrics MSE, MAE, RMSE on the predictions.

14. Plot train, test, and predictions in a single plot.

AIM

To analyze monthly airline passenger data, check stationarity, build an ARIMA forecasting model, and evaluate its prediction accuracy.

ALGORITHM

1. Load the airline passenger dataset and explore its contents.
2. Plot the time series to identify trends and seasonality.
3. Perform the ADF test to check stationarity.
4. Apply differencing or transformations if required.
5. Split the data into training and testing sets.
6. Fit an ARIMA model and forecast passenger numbers.
7. Evaluate forecasts using MSE, MAE, and RMSE.
8. Plot training data, testing data, and predictions.

PROGRAM AND OUTPUT

Question 1: Download the airline-passengers dataset from the given URL.

```
# LUQMAN T (241801141)

import pandas as pd

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
df = pd.read_csv(url)
print(df.head())
```

```
     Month  Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121
```

Question 2: Read the CSV using parse_dates and index_col so the index becomes a DateTimeIndex.

```
# LUQMAN T (241801141)

df = pd.read_csv(url, parse_dates=["Month"], index_col="Month")
print(df.index)
print(df.head())
```

```
DatetimeIndex(['1949-01-01', '1949-02-01', '1949-03-01', '1949-04-01',
               '1949-05-01', '1949-06-01', '1949-07-01', '1949-08-01',
               '1949-09-01', '1949-10-01',
               ...
               '1960-03-01', '1960-04-01', '1960-05-01', '1960-06-01',
               '1960-07-01', '1960-08-01', '1960-09-01', '1960-10-01',
               '1960-11-01', '1960-12-01'],
              dtype='datetime64[ns]', name='Month', length=144, freq=None)
            Passengers
Month
1949-01-01         112
1949-02-01         118
1949-03-01         132
1949-04-01         129
1949-05-01         121
```

Question 3: Explore the dataset.

```
# LUQMAN T (241801141)

print(df.info())
print(df.describe())
print(df.isnull().sum())
```

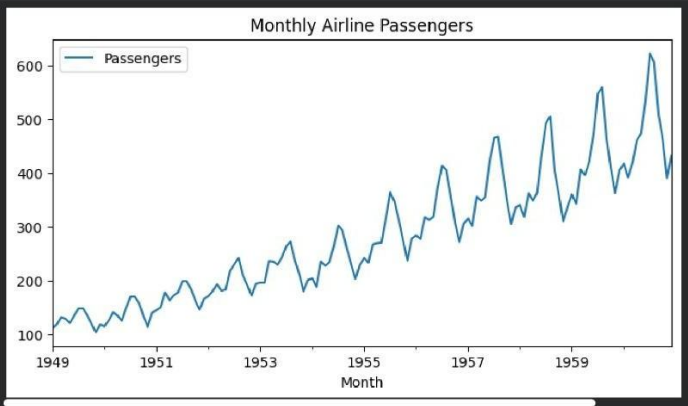
```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 144 entries, 1949-01-01 to 1960-12-01
Data columns (total 1 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Passengers  144 non-null    int64
dtypes: int64(1)
memory usage: 2.2 KB
None
       Passengers
count  144.000000
mean   280.298611
std    119.966317
min    104.000000
25%    180.000000
50%    265.500000
75%    360.500000
max    622.000000
Passengers    0
dtype: int64
```

Question 4: Plot the series to inspect trend, seasonality, and outliers.

```
# LUQMAN T (241801141)

import matplotlib.pyplot as plt

df.plot(figsize=(8, 4))
plt.title("Monthly Airline Passengers")
plt.show()
```



Question 5: Check stationarity using the Augmented Dickey-Fuller (ADF) test.

```
# LUQMAN T (241801141)

from statsmodels.tsa.stattools import adfuller

result = adfuller(df.iloc[:, 0])
print("ADF Statistic:", result[0])
print("p-value:", result[1])
```

```
ADF Statistic: 0.8153688792060498
p-value: 0.991880243437641
/tmp/ipykernel_738/2757683927.py:5: FutureWarning: adfuller currently returns a plain tuple whose length depends on the sto
  result = adfuller(df.iloc[:, 0])
```

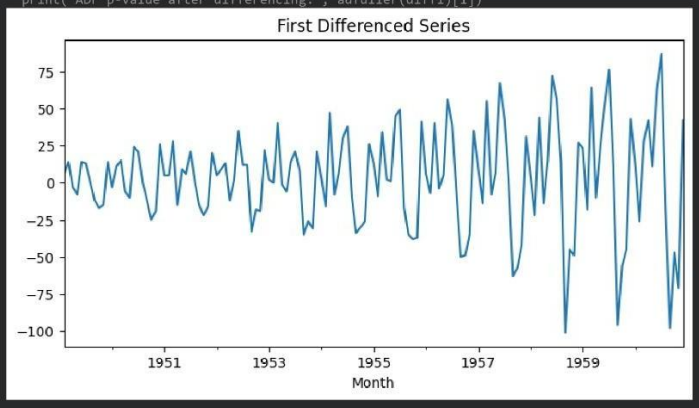
Question 6: If non-stationary, difference the series to make it stationary.

```
# LUQMAN T (241801141)

series = df.iloc[:, 0]
diff1 = series.diff().dropna()

print("ADF p-value after differencing:", adfuller(diff1)[1])
diff1.plot(figsize=(8, 4), title="First Differenced Series")
plt.show()
```

```
ADF p-value after differencing: 0.05421329028382478
/tmp/ipykernel_738/251278962.py:6: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stor
  print("ADF p-value after differencing:", adfuller(diff1)[1])
```



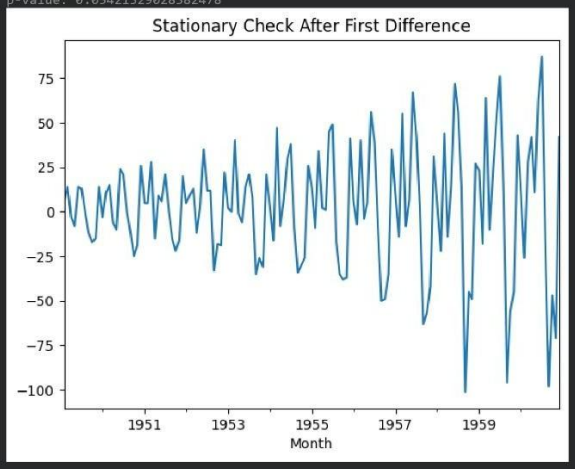
Question 7: Re-run the ADF test on the differenced series and visualize it.

```
# LUQMAN T (241801141)

adf = adfuller(diff1)
print("ADF Statistic:", adf[0])
print("p-value:", adf[1])

diff1.plot(title="Stationary Check After First Difference")
plt.show()
```

```
/tmp/ipykernel_738/1424509867.py:3: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stor
  adf = adfuller(diff1)
ADF Statistic: -2.8292668241700047
p-value: 0.05421329028382478
```



Question 8: If still non-stationary, try second differencing or a log transformation.

```
# LUQMAN T (241801141)

diff2 = series.diff().diff().dropna()
print("Second Difference ADF p-value:", adfuller(diff2)[1])

log_diff = __import__("numpy").log(series).diff().dropna()
print("Log Difference ADF p-value:", adfuller(log_diff)[1])
```

```
Second Difference ADF p-value: 2.7328918500142026e-29
Log Difference ADF p-value: 0.07112054815086184
/tmp/ipykernel_738/1237123172.py:4: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stor
  print("Second Difference ADF p-value:", adfuller(diff2)[1])
/tmp/ipykernel_738/1237123172.py:7: FutureWarning: adfuller currently returns a plain tuple whose length depends on the stor
  print("Log Difference ADF p-value:", adfuller(log_diff)[1])
```

Question 9: Split the original series into training and testing sets.

```
# LUQMAN T (241801141)

train = series.iloc[:-12]
test = series.iloc[-12:]

print("Train:", len(train))
print("Test:", len(test))
```

```
Train: 132
Test: 12
```

Question 10: Fit an ARIMA model on the training set using the differencing order selected above.

```
# LUQMAN T (241801141)

from statsmodels.tsa.arima.model import ARIMA

d = 1
model = ARIMA(train, order=(2, d, 2))
fit = model.fit()
print(fit.summary())
```

```
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was pr
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was pr
  self._init_dates(dates, freq)
/usr/local/lib/python3.13/dist-packages/statsmodels/tsa/base/tsa_model.py:480: ValueWarning: No frequency information was pr
  self._init_dates(dates, freq)
                               SARIMAX Results
==============================================================================
Dep. Variable:            Passengers   No. Observations:                  132
Model:                 ARIMA(2, 1, 2)   Log Likelihood                -607.782
Date:                Tue, 13 Oct 2026   AIC                           1225.563
Time:                        13:45:10   BIC                           1239.939
Sample:                    01-01-1949   HQIC                          1231.405
                         - 12-01-1959
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1          1.6660      0.024     68.561      0.000       1.618       1.714
ar.L2         -0.9294      0.022    -41.536      0.000      -0.973      -0.886
ma.L1         -1.8324      0.032    -56.857      0.000      -1.896      -1.769
ma.L2          0.9581      0.032     29.998      0.000       0.895       1.021
sigma2       605.1588     73.622      8.220      0.000     460.863     749.454
===================================================================================
Ljung-Box (L1) (Q):                   0.45   Jarque-Bera (JB):                 0.78
Prob(Q):                              0.50   Prob(JB):                         0.68
Heteroskedasticity (H):               6.52   Skew:                             0.19
Prob(H) (two-sided):                  0.00   Kurtosis:                         3.01
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

Question 11: Forecast the test dataset.

```
# LUQMAN T (241801141)

pred = fit.forecast(steps=len(test))
print(pred)
```

```
1960-01-01    411.312453
1960-02-01    430.812660
1960-03-01    457.433605
```

```
1960-04-01    483.661492
1960-05-01    502.616754
1960-06-01    509.821046
1960-07-01    504.207054
1960-08-01    488.158647
1960-09-01    466.639307
1960-10-01    445.702753
1960-11-01    430.821634
1960-12-01    425.487308
Freq: MS, Name: predicted_mean, dtype: float64
```

Question 12: Calculate MSE, MAE, and RMSE for the forecasts.

```
# LUQMAN T (241801141)

import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error

mse = mean_squared_error(test, pred)
mae = mean_absolute_error(test, pred)
rmse = np.sqrt(mse)

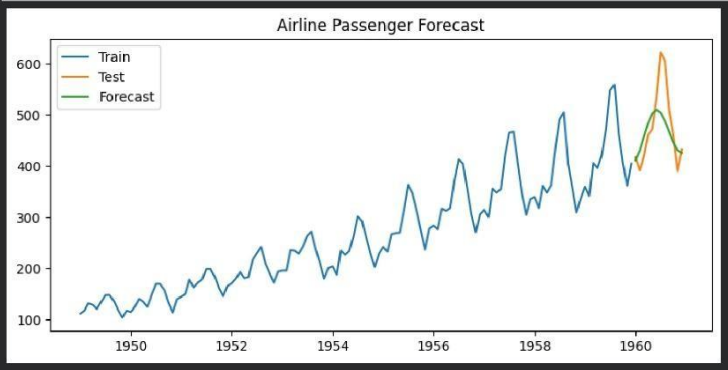
print("MSE:", mse)
print("MAE:", mae)
print("RMSE:", rmse)
```

```
MSE: 3049.561918715339
MAE: 41.83479799137674
RMSE: 55.2228387419131
```

Question 13: Plot training data, testing data, and predictions together.

```
# LUQMAN T (241801141)

plt.figure(figsize=(9, 4))
plt.plot(train, label="Train")
plt.plot(test, label="Test")
plt.plot(pred, label="Forecast")
plt.title("Airline Passenger Forecast")
plt.legend()
plt.show()
```



RESULT

The airline passenger time series was analyzed, stationarity was tested, and an ARIMA model was used to forecast passenger numbers. Forecast accuracy was evaluated using MSE, MAE, and RMSE.